```
action_set = {
    0: 'u',
    1: 'd',
    2: 'l',
    3: 'r',
}
```

Okay, let’s get to coding the main training loop.

**Listing 3.3. Q-learning: Main training loop**

```
epochs = 1000
losses = []                                                    1
for i in range(epochs):                                        2
    game = Gridworld(size=4, mode='static')                    3
    state_ = game.board.render_np().reshape(1,64) \
             + np.random.rand(1,64)/10.0                       4
    state1 = torch.from_numpy(state_).float()                  5
    status = 1                                                 6
    while(status == 1):                                        7
        qval = model(state1)                                   8
        qval_ = qval.data.numpy()
        if (random.random() < epsilon):                        9
            action_ = np.random.randint(0,4)
        else:
            action_ = np.argmax(qval_)

        action = action_set[action_]                           10
        game.makeMove(action)                                  11
        state2_ = game.board.render_np().reshape(1,64) +
     np.random.rand(1,64)/10.0
        state2 = torch.from_numpy(state2_).float()             12
        reward = game.reward()
        with torch.no_grad():
            newQ = model(state2.reshape(1,64))
        maxQ = torch.max(newQ)                                 13
        if reward == -1:                                       14
            Y = reward + (gamma * maxQ)
        else:
            Y = reward
        Y = torch.Tensor([Y]).detach()
        X = qval.squeeze()[action_]                            15
        loss = loss_fn(X, Y)
        optimizer.zero_grad()
        loss.backward()
        losses.append(loss.item())
        optimizer.step()
        state1 = state2
        if reward != -1:                                       16
```